



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

واحد کنترل ریز برنامه سازی شده

نام درس

آزمایشگاه معماری کامپیوتر

نیم سال ۱۴۰۴۳

استاد

دکتر سربازی آزاد

گروه ۲

امیرشایان سلیمانی نیا - ۴۰۳۱۰۶۱۰۸

محمد مهدی مرادی - ۴۰۳۱۰۶۶۸۱

فهرست مطالب

۱	هدف آزمایش	۲
۲	روند توسعه معماری در آزمایش های ششم تا نهم	۲
۳	قالب دستورهای ماشین باقی مانده از آزمایش هشتم	۲
۱.۳	دستورهای محاسباتی و انتقالی	۳
۲.۳	دستورهای دسترسی به حافظه داده	۳
۳.۳	دستورهای پرش	۳
۴	قالب ریزدستور	۳
۵	ساختار مدار و زیرمدارها	۵
۱.۵	زیرمدارهای بدون تغییر	۵
۲.۵	طراحی واحد کنترل ریزبرنامه ای	۷
۳.۵	مدار اصلی	۹
۶	ریزبرنامه واکشی و اجرای دستورها	۹
۱.۶	مرحله واکشی و تشخیص نوع دستور	۱۰
۲.۶	اجرای دستورهای محاسباتی و حافظه	۱۰
۳.۶	اجرای دستورهای پرش	۱۰
۷	تست و شبیه سازی	۱۱
۱.۷	تست اول: مجموع ده جمله اول سری فیبوناچی	۱۱
۲.۷	تست دوم: جمع دو عدد ۴۸ بیتی	۱۲
۸	نتیجه گیری	۱۳

۱ هدف آزمایش

هدف آزمایش نهم، طراحی و پیاده‌سازی واحد کنترل کامپیوتر ساده ساخته شده در آزمایش‌های ششم، هفتم و هشتم به صورت ریزبرنامه‌پذیر است. در آزمایش هشتم، سیگنال‌های کنترلی با استفاده از منطق سخت‌سیمی تولید می‌شدند؛ اما در این آزمایش، توالی واکنشی و اجرای دستورهای ماشین به شکل مجموعه‌ای از ریزدستورها در یک ریزحافظه ذخیره می‌شود.

مطابق دستورکار، ریزحافظه دارای ۲۵۶ کلمه است و قالب هر ریزدستور باید علاوه بر صدور سیگنال‌های کنترلی، هشت نوع ادامه اجرای ریزبرنامه را پشتیبانی کند. پس از نوشتن ریزبرنامه‌های واکنشی و اجرای تمام گروه‌های دستور، دو برنامه مجموع ده جمله اول سری فیبوناچی و جمع دو عدد چندبایتی، که در آزمایش هشتم برای ارزیابی پردازنده استفاده شده بودند، دوباره با واحد کنترل ریزبرنامه‌ای اجرا می‌شوند.

طراحی و شبیه‌سازی مدار در محیط Logisim-evolution 3.9.0 انجام شده است. برای اطمینان از صحت عملکرد، وضعیت ریزشمارنده، سیگنال‌های کنترلی، حافظه برنامه و حافظه داده در اجرای مرحله به مرحله بررسی شدند.

۲ روند توسعه معماری در آزمایش‌های ششم تا نهم

آزمایش نهم یک طراحی مستقل نیست، بلکه آخرین مرحله از روند ساخت کامپیوتر ساده در چهار آزمایش متوالی است. جدول ۱ نقش هر مرحله را نشان می‌دهد.

آزمایش	قابلیت افزوده شده
ششم	طراحی واحد محاسبات، چهار ثبات عمومی هشت‌بیتی و اجرای دستورهای شش‌بیتی جمع، تفریق و انتقال با امکان انتخاب ثبات مبدأ و مقصد.
هفتم	افزودن شمارنده برنامه، حافظه برنامه و ثبات دستور؛ در نتیجه دستورها به صورت ترتیبی از حافظه واکنشی و اجرا شدند.
هشتم	افزودن حافظه داده ۳۲ بیتی، ثبات وضعیت، دستورات خواندن و نوشتن حافظه و پرش‌های شرطی و غیرشرطی؛ در این مرحله معماری هاروارد کامل شد.
نهم	حفظ مسیر داده و مجموعه دستورهای آزمایش هشتم و جایگزینی واحد کنترل سخت‌سیمی با واحد کنترل ریزبرنامه‌ای مبتنی بر μPC ، ریزحافظه و μIR .

جدول ۱: روند تکمیل کامپیوتر ساده از آزمایش ششم تا نهم

بنابراین تغییر اصلی این آزمایش فقط در زیرمدار CU انجام شده است. قالب دستورهای هشت‌بیتی، حافظه‌های برنامه و داده، فایل ثبات، واحد محاسبات، ثبات وضعیت و زیرمدار تجزیه دستور همان ساختار آزمایش هشتم را دارند. این ویژگی امکان مقایسه مستقیم کنترل سخت‌سیمی و ریزبرنامه‌ای را فراهم می‌کند؛ زیرا رفتار قابل مشاهده پردازنده ثابت مانده و تنها روش تولید سیگنال‌های کنترلی تغییر کرده است.

۳ قالب دستورهای ماشین باقی مانده از آزمایش هشتم

برای نوشتن ریزبرنامه اجرای دستورها باید سه قالب دستور ماشین آزمایش هشتم در نظر گرفته شوند. دو بیت بالایی دستور، گروه کلی آن را مشخص می‌کنند.

۱.۳ دستورهای محاسباتی و انتقالی

در این گروه دو بیت بالایی برابر 00 هستند. بیت بعدی نوع عملیات، دو بیت میانی ثبات مقصد و سه بیت پایینی منبع عملوند دوم را تعیین می‌کند. واحد محاسبات عملیات جمع یا تفریق را انجام داده و نتیجه را در یکی از ثبات‌های R_0 تا R_3 قرار می‌دهد.

بیت‌ها	میدان	معنا
b_7b_6	Opcode	00: دستور محاسباتی یا انتقالی
b_5	ALU Function	صفر: جمع، یک: تفریق
b_4b_3	Destination	انتخاب یکی از R_0 تا R_3
$b_2b_1b_0$	Source	یکی از ثبات‌ها یا ثبات‌های صفر، یک و منفی یک

جدول ۲: قالب دستورهای محاسباتی و انتقالی

۲.۳ دستورهای دسترسی به حافظه داده

در دستورهای حافظه، دو بیت بالایی برابر 01 هستند. بیت پنجم نوع دسترسی را مشخص می‌کند و پنج بیت پایین آدرس یکی از ۳۲ خانه حافظه داده‌اند. در عملیات خواندن، محتوای خانه انتخاب شده در R_0 قرار می‌گیرد و در عملیات نوشتن، محتوای R_0 در حافظه ذخیره می‌شود.



۳.۳ دستورهای پرش

دو بیت بالایی دستور پرش برابر 11 هستند. دو بیت بعدی شرط پرش و چهار بیت کم‌ارزش آدرس مقصد در حافظه برنامه را مشخص می‌کنند. شرط‌ها شامل پرش در صورت فعال بودن پرچم‌های صفر، علامت و سرریز و همچنین پرش بدون شرط هستند.

شرط پرش	عملکرد
00	پرش در صورت $Z = 1$
01	پرش در صورت $S = 1$
10	پرش در صورت $O = 1$
11	پرش بدون شرط

جدول ۳: کدگذاری شرط دستورهای پرش

۴ قالب ریزدستور

ریزدستورهای واحد کنترل یازده‌بیتی هستند. سه بیت بالایی، میدان Opcode، نوع ادامه اجرای ریزبرنامه را تعیین می‌کنند و هشت بیت پایین بسته به نوع ریزدستور، سیگنال‌های کنترلی یا آدرس مقصد پرش را در

خود دارند:

$$\underbrace{\mu Op_2 \mu Op_1 \mu Op_0}_{\text{سه بیت}} \quad \underbrace{D_7 D_6 D_5 D_4 D_3 D_2 D_1 D_0}_{\text{هشت بیت}}$$

معنای هشت مقدار ممکن میدان $\mu Opcode$ مطابق دستورکار و جدول ۴ است. در نام‌گذاری مدار، IR0، IR1 و IR2 به ترتیب به بیت‌های ۵، ۶ و ۷ ثبات دستور متصل شده‌اند.

عملکرد	مقدار $\mu Opcode$
اجرای عادی و رفتن به ریزدستور بعدی	000
پرش در صورت IR0=1	001
پرش در صورت IR1=1	010
پرش در صورت IR2=1	011
پرش در صورت Z = 1	100
پرش در صورت S = 1	101
پرش در صورت O = 1	110
پرش بدون شرط	111

جدول ۴: هشت نوع ریزدستور موردنیاز در آزمایش نهم

وقتی $\mu Opcode=000$ باشد، هشت بیت پایین به عنوان بردار سیگنال‌های کنترلی تفسیر می‌شوند. جدول ۵ نگاشت این بیت‌ها را نشان می‌دهد. بیت صفر در پیاده‌سازی حاضر استفاده نشده است.

بیت	سیگنال	وظیفه
۷	En_PC	افزایش شمارنده برنامه برای واکنشی دستور بعدی
۶	Load_PC	بارگذاری آدرس مقصد دستور پرش در PC
۵	Load_IR	ذخیره خروجی حافظه برنامه در IR
۴	OE	فعال کردن بافر سه‌حالت خروجی ALU
۳	En_Read	فعال کردن خواندن از حافظه داده
۲	En_Write	فعال کردن نوشتن در حافظه داده
۱	Load_SR	ذخیره نتیجه و پرچم‌ها در مسیر داده و ثبات وضعیت
۰	—	استفاده نشده

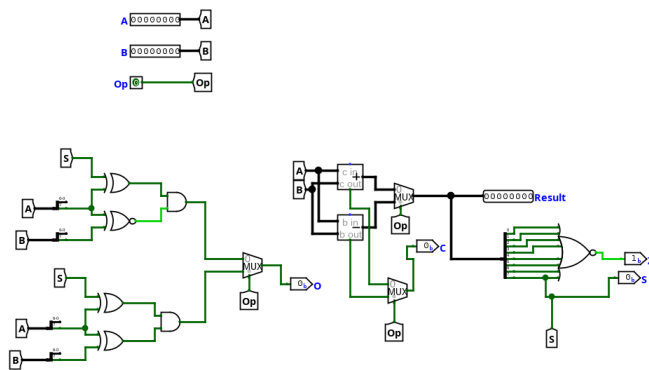
جدول ۵: نگاشت هشت بیت پایین در ریزدستور عادی

در ریزدستورهای پرش، هشت بیت پایین آدرس مقصد در ریزحافظه هستند. در این حالت نباید همان بیت‌ها به اشتباه سیگنال‌های کنترلی را فعال کنند؛ به همین دلیل، مدار صدور سیگنال‌ها فقط در حالت $\mu Opcode=000$ بردار پایین ریزدستور را به خروجی متصل می‌کند و در سایر حالت‌ها تمام خروجی‌های کنترلی را صفر نگه می‌دارد.

۵ ساختار مدار و زیرمدارها

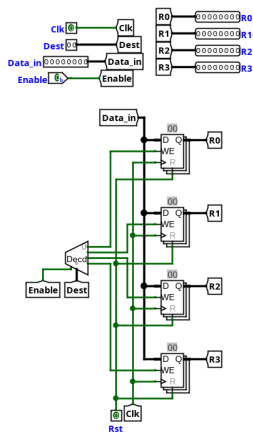
۱.۵ زیرمدارهای بدون تغییر

چهار زیرمدار ALU، Reg_File، DP و Unpacker نسبت به آزمایش هشتم تغییر نکرده‌اند؛ زیرا مجموعه دستورهای ماشین و مسیر داده ثابت مانده‌اند. در ادامه، وظیفه هر بخش به اختصار بیان شده است. زیرمدار ALU جمع و تفریق هشت‌بیتی را انجام داده و پرچم‌های نقلی، صفر، علامت و سرریز را تولید می‌کند. انتخاب نتیجه جمع یا تفریق توسط سیگنال Op انجام می‌شود.



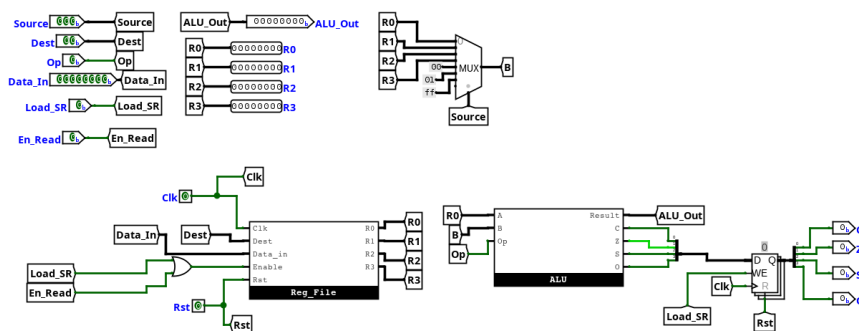
شکل ۱: ساختار زیرمدار ALU

زیرمدار Reg_File شامل چهار ثبات عمومی هشت‌بیتی R_0 تا R_3 است. دیگر، با توجه به میدان مقصد و سیگنال فعال‌ساز، نوشتن در ثبات انتخاب‌شده را کنترل می‌کند.



شکل ۲: ساختار زیرمدار Reg_File

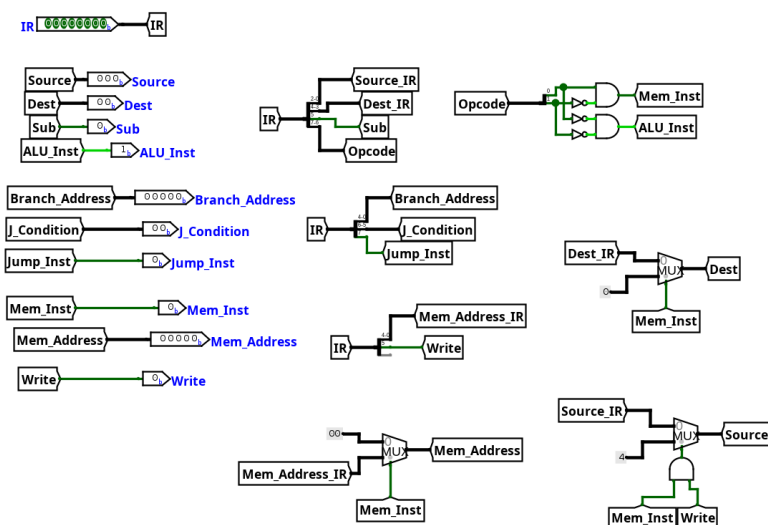
زیرمدار DP فایل ثبات و واحد محاسبات را به یکدیگر متصل می‌کند. انتخاب عملوندها، نوشتن نتیجه در ثبات مقصد و ذخیره پرچم‌های C ، Z ، S و O در ثبات وضعیت در این بخش انجام می‌شود.



شکل ۳: ساختار زیرمدار مسیر داده یا DP

زیرمدار Unpacker دستور هشت‌بیتی موجود در IR را بر اساس سه قالب دستور تجزیه می‌کند و میدان‌هایی مانند مبدأ، مقصد، نوع عملیات، آدرس حافظه، آدرس پرش و شرط پرش را در اختیار سایر

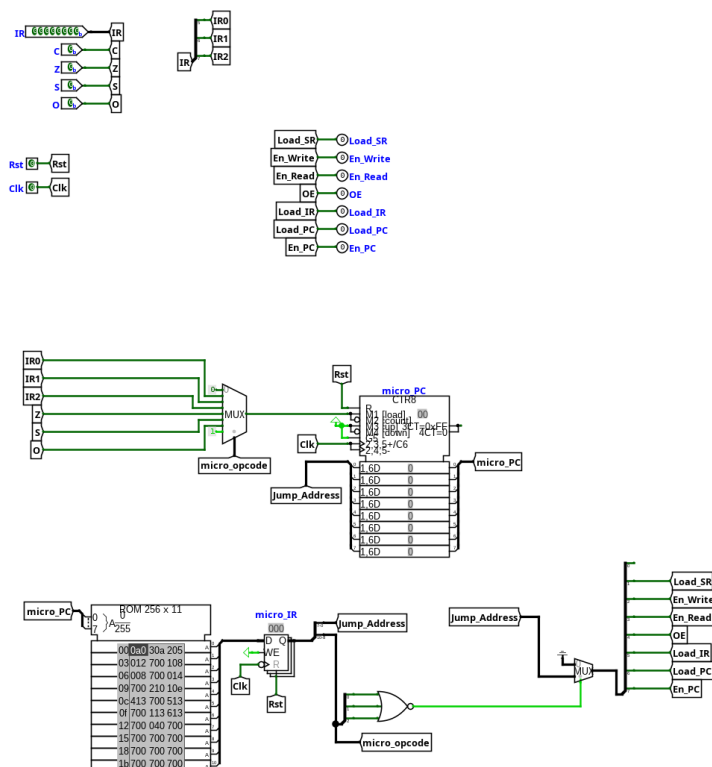
قسمت‌ها قرار می‌دهد.



شکل ۴: ساختار زیرمدار تجزیه دستور یا Unpacker

۲.۵ طراحی واحد کنترل ریزبرنامه‌ای

تغییر اصلی آزمایش در زیرمدار CU انجام شده است. این واحد ورودی‌های IR، پرچم‌های وضعیت، کلاک و ریست را دریافت کرده و سیگنال‌های $Load_PC$ ، $Load_IR$ ، OE ، En_Read ، En_Write ، $Load_SR$ و En_PC را تولید می‌کند. چون رابط ورودی و خروجی واحد کنترل حفظ شده است، جایگزینی آن با CU آزمایش هشتم بدون تغییر در مدارهای دیگر انجام شد.



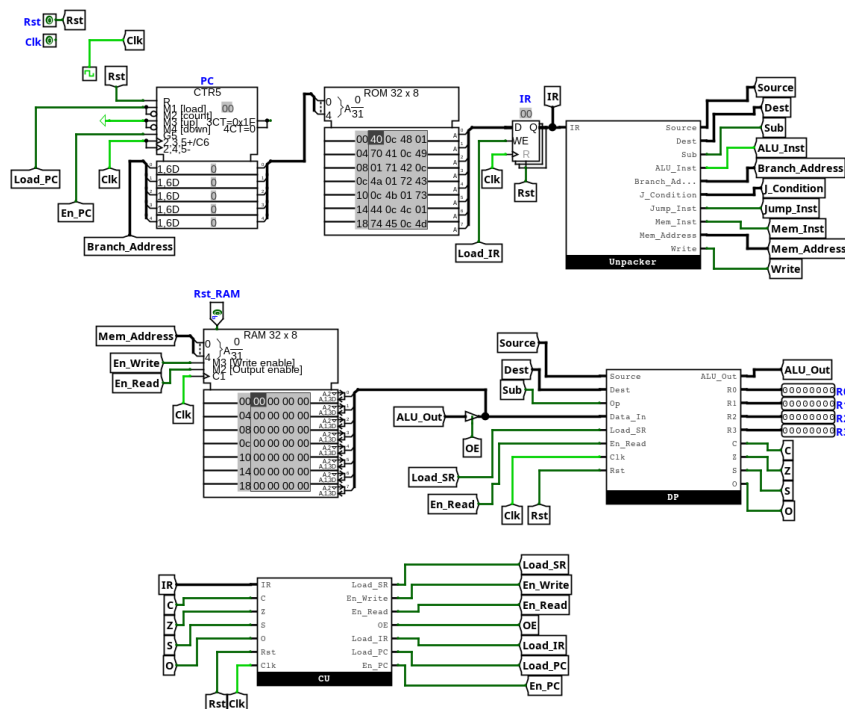
شکل ۵: ساختار کامل واحد کنترل ریزبرنامه‌ای

اجزای اصلی شکل ۵ عبارت‌اند از:

- شمارنده ریزبرنامه یا μPC : یک شمارنده هشت‌بیتی که آدرس یکی از ۲۵۶ خانه ریزحافظه را تولید می‌کند. در حالت عادی افزایش می‌یابد و در صورت برقراردن شرط پرش، آدرس `Jump_Address` را بارگذاری می‌کند.
- ریزحافظه: یک $ROM\ 11 \times 256$ که ریزبرنامه‌های واکشی و اجرای دستورها را نگه می‌دارد.
- ثبات ریزدستور یا μIR : ثباتی یازده‌بیتی که خروجی ریزحافظه را در لبه پایین‌رونده کلاک ذخیره می‌کند. در نتیجه ریزدستور پیش از لبه فعال اجزای مسیر داده پایدار می‌شود.
- مالتی‌پلکسر شرط: با استفاده از میدان $\mu Opcode$ یکی از مقادیر صفر، $IR0$ ، $IR1$ ، $IR2$ ، Z ، S ، O یا یک را انتخاب می‌کند. خروجی این مالتی‌پلکسر ورودی بارگذاری μPC است.
- مدار دروازه‌بندی خروجی: سه بیت $\mu Opcode$ وارد گیت NOR می‌شوند. فقط در ریزدستور عادی، مالتی‌پلکسر خروجی اجازه می‌دهد هشت بیت پایین به سیگنال‌های کنترل تبدیل شوند؛ در ریزدستور پرش، بردار صفر به خروجی فرستاده می‌شود.

پس از ریست، μPC صفر می‌شود و اجرای ریزبرنامه از مرحله واکشی آغاز می‌گردد. در هر مسیر اجرایی، پس از تکمیل عملیات دستور ماشین، یک ریزدستور پرش بدون شرط کنترل را دوباره به آدرس صفر ریزحافظه باز می‌گرداند.

در مدار اصلی، معماری هاروارد آزمایش هشتم حفظ شده است؛ بنابراین حافظه برنامه و حافظه داده از یکدیگر جدا هستند. شمارنده برنامه آدرس ROM برنامه را تولید می‌کند، دستور در IR ذخیره و توسط Unpacker تجزیه می‌شود. واحد کنترل ریزبرنامه‌ای با دریافت دستور و پرچم‌های وضعیت، توالی سیگنال‌های لازم برای مسیر داده، حافظه داده و شمارنده برنامه را صادر می‌کند.



شکل ۶: مدار اصلی کامپیوتر با واحد کنترل ریزبرنامه‌ای

۶ ریزبرنامه واکشی و اجرای دستورها

محتوای ریزحافظه از آدرس 0x00 تا 0x14 در جدول ۶ تحلیل شده است. تمام خانه‌های بعدی با کلمه 0x700 پر شده‌اند. این ریزدستور یک پرش بدون شرط به آدرس صفر است؛ بنابراین در صورت ورود به هر خانه استفاده نشده، کنترل به مرحله واکشی بازمی‌گردد.

جدول ۶: محتوای ریزحافظه و نقش هر ریزدستور

آدرس	ریزدستور	عملکرد
0x00	0x0A0	فعال کردن Load_IR و En_PC برای واکشی دستور و افزایش شمارنده برنامه.
0x01	0x30A	اگر IR2=1 باشد، دستور از نوع پرش است و ریزبرنامه به آدرس 0x0A می‌رود.
0x02	0x205	اگر IR1=1 باشد، دستور از نوع حافظه است و ریزبرنامه به آدرس 0x05 می‌رود؛ در غیر این صورت مسیر دستور محاسباتی ادامه می‌یابد.
0x03	0x012	فعال کردن OE و Load_SR برای اجرای دستور محاسباتی، نوشتن نتیجه و ذخیره پرچم‌ها.

آدرس	ریزدستور	عملکرد
0x04	0x700	بازگشت بدون شرط به مرحله واکشی.
0x05	0x108	بررسی بیت IR0؛ در صورت یک بودن، دستور نوشتن است و مسیر آدرس 0x08 انتخاب می شود.
0x06	0x008	فعال کردن En_Read و بارگذاری داده حافظه در R ₀ .
0x07	0x700	بازگشت بدون شرط به مرحله واکشی.
0x08	0x014	فعال کردن OE و En_Write برای ذخیره محتوای R ₀ در حافظه داده.
0x09	0x700	بازگشت بدون شرط به مرحله واکشی.
0x0A	0x210	بررسی IR1 برای تفکیک حالت های دستور پرش.
0x0B	0x10E	بررسی IR0؛ در صورت یک بودن، ادامه از آدرس 0x0E.
0x0C	0x413	بررسی پرچم Z و انتقال به مرحله بارگذاری PC در صورت برقرار بودن شرط.
0x0D	0x700	بازگشت به واکشی در صورت برقرار نبودن شرط پرش.
0x0E	0x513	بررسی پرچم S و انتقال به مرحله بارگذاری PC.
0x0F	0x700	بازگشت به واکشی در صورت برقرار نبودن شرط پرش.
0x10	0x113	بررسی IR0؛ یک بودن آن در این مسیر نشان دهنده پرش بدون شرط است.
0x11	0x613	بررسی پرچم O و انتقال به مرحله بارگذاری PC.
0x12	0x700	بازگشت به واکشی در صورت برقرار نبودن شرط پرش.
0x13	0x040	فعال کردن Load_PC و بارگذاری آدرس مقصد موجود در دستور.
0x14	0x700	بازگشت بدون شرط به مرحله واکشی.

۱.۶ مرحله واکشی و تشخیص نوع دستور

در آدرس صفر، سیگنال های Load_IR و En_PC هم زمان فعال می شوند؛ در نتیجه دستور جاری وارد IR شده و PC برای دستور بعدی افزایش می یابد. سپس بیت های بالایی IR به صورت شاخه ای بررسی می شوند. یک بودن بیت ۷، مسیر دستور پرش را انتخاب می کند و در غیر این صورت بیت ۶ میان دستور حافظه و دستور محاسباتی تمایز ایجاد می کند.

۲.۶ اجرای دستورهای محاسباتی و حافظه

در مسیر محاسباتی، ریزدستور 0x012 خروجی ALU را روی باس قرار داده و سیگنال Load_SR را فعال می کند. به این ترتیب نتیجه در ثبات مقصد و پرچم ها در ثبات وضعیت ذخیره می شوند. در مسیر حافظه، بیت نوشتن بررسی می شود. برای خواندن، سیگنال En_Read فعال می شود تا محتوای حافظه در R₀ قرار گیرد. برای نوشتن، OE و En_Write هم زمان فعال می شوند تا مقدار R₀ روی باس قرار گرفته و در آدرس تعیین شده ذخیره شود.

۳.۶ اجرای دستورهای پرش

دو بیت شرط پرش، ریزبرنامه را به بررسی یکی از پرچم های Z، S و O یا مسیر پرش بدون شرط هدایت می کنند. اگر شرط برقرار باشد، کنترل به آدرس 0x13 می رود و Load_PC فعال می شود. اگر شرط برقرار نباشد، شمارنده برنامه تغییر نمی کند و ریزبرنامه به مرحله واکشی دستور بعدی بازمی گردد.

۷ تست و شبیه‌سازی

برای بررسی صحت واحد کنترل ریزبرنامه‌ای، برنامه‌های آزمایش هشتم بدون تغییر در قالب دستورهای ماشین اجرا شدند. در این حالت، هر دستور ماشین طی چند ریزگام واکشی، تشخیص و اجرا می‌شود؛ بنابراین درست بودن نتایج نشان می‌دهد که ریزبرنامه قادر است همهٔ سیگنال‌های موردنیاز مسیر داده، حافظه و پرش را با ترتیب صحیح تولید کند.

۱.۷ تست اول: مجموع ده جملهٔ اول سری فیبوناچی

برنامهٔ نخست مجموع ده جملهٔ اول سری فیبوناچی را با استفاده از حلقه محاسبه کرده و در آدرس صفر حافظهٔ داده ذخیره می‌کند. مجموع مورد انتظار برابر است با:

$$0 + 1 + 1 + 2 + 3 + 5 + 8 + 13 + 21 + 34 = 88 = 0x58.$$

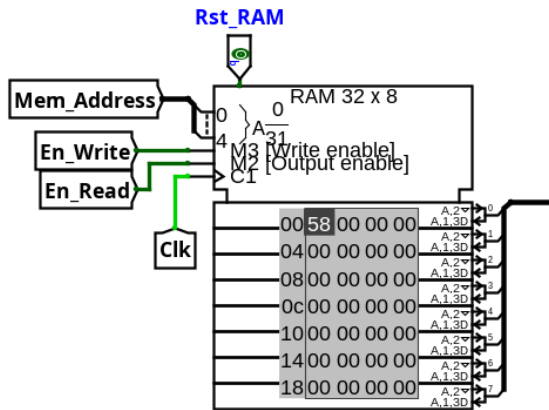
کد ماشین قرارگرفته در حافظهٔ برنامه به‌صورت زیر است:

```
00: 20 14 05 0c 20 05 00 00 00 05 05 1c 20 01 02 32
10: 0c 20 03 25 1c 99 04 ec 04 20 01 25 60 fd 04 00
```

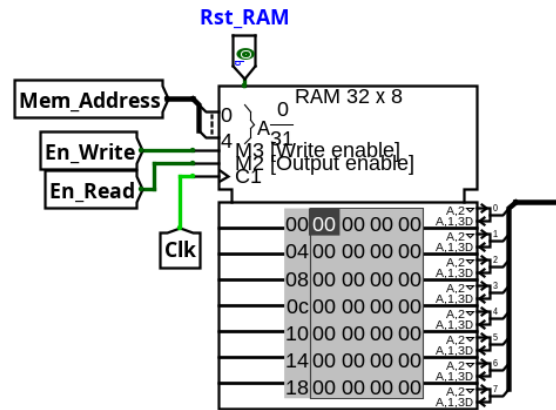
ابتدا مقادیر صفر و یک دنباله در R_2 و R_1 قرار می‌گیرند و R_3 با مقدار ده نقش شمارندهٔ حلقه را دارد. در هر تکرار، جملهٔ جدید با جمع دو جملهٔ قبلی ساخته شده و مقادیر ثبات‌ها به‌روزرسانی می‌شوند. سپس شمارندهٔ حلقه یک واحد کم می‌شود. دستور 0x99 با استفاده از پرچم صفر خروج از حلقه را کنترل می‌کند و دستور 0xEC در صورت پایان‌نیافتن حلقه، اجرای برنامه را به ابتدای آن بازمی‌گرداند.

پس از ده تکرار، مقدار نهایی در R_0 ساخته و با دستور 0x60 در آدرس صفر حافظهٔ داده ذخیره می‌شود. دستور 0xFD نیز یک پرش بدون شرط به آدرس خودش است و مطابق خواستهٔ دستورکار، پردازنده را پس از پایان محاسبه متوقف نگه می‌دارد.

در شکل ۷، حافظه پیش از اجرای برنامه کاملاً صفر است. پس از اجرای برنامه، مقدار 0x58 در آدرس 0x00 قرار گرفته است. این مقدار در مبنای ده برابر ۸۸ است و با مجموع محاسبه‌شده مطابقت دارد.



(ب) ذخیره مقدار 0x58 در آدرس صفر



(آ) وضعیت حافظه پیش از اجرای برنامه

شکل ۷: مقایسه حافظه پیش و پس از اجرای برنامه مجموع فیونانچی

۲.۷ تست دوم: جمع دو عدد ۴۸ بیتی

در دستور کار، جمع دو عدد ۶۴ بیتی ذخیره شده از آدرس های صفر و هشت و ذخیره حاصل از آدرس شانزده خواسته شده است. با این حال، حافظه برنامه مدار تنها ۳۲ کلمه هشت بیتی ظرفیت دارد. در روش مستقیم استفاده شده، جمع هر بایت به پنج دستور ماشین نیاز دارد: خواندن بایت اول، کپی آن در R_1 ، خواندن بایت دوم، جمع دو مقدار و ذخیره حاصل.

بنابراین اجرای مستقیم جمع هشت بایت به همراه دستور توقف به

$$8 \times 5 + 2 = 42$$

خانه نیاز دارد و در حافظه برنامه جا نمی شود. برنامه به شش بایت، یعنی ۴۸ بیت، محدود شد؛ زیرا

$$6 \times 5 + 2 = 32$$

دقیقاً تمام ظرفیت حافظه برنامه را مصرف می کند. کد ماشین برنامه به صورت زیر است:

00: 40 0c 48 01 70 41 0c 49 01 71 42 0c 4a 01 72 43

10: 0c 4b 01 73 44 0c 4c 01 74 45 0c 4d 01 75 fe 04

در هر بلوک پنج دستوری، یک بایت از عدد اول از آدرس های 0x00 تا 0x05 و بایت متناظر عدد دوم از آدرس های 0x08 تا 0x0D خوانده می شود. حاصل در آدرس های 0x10 تا 0x15 ذخیره می گردد. دستور 0xFE در انتهای برنامه، با پرش بدون شرط به آدرس خودش، پردازنده را متوقف نگه می دارد.

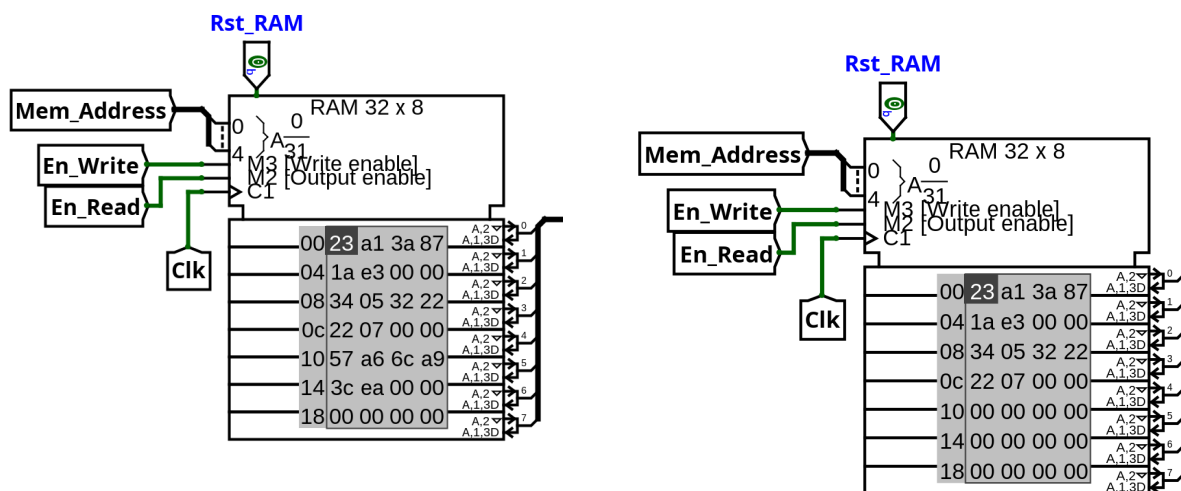
در تصویر پیش از اجرا، دو عدد زیر در حافظه قرار دارند و ناحیه نتیجه از آدرس 0x10 صفر است:

$$A = 0x23A13A871AE3, \quad B = 0x340532222207.$$

پس از اجرا، شش بایت EA 3C A9 6C A6 57 از آدرس 0x10 ذخیره شده اند. بنابراین:

$$0x23A13A871AE3 + 0x340532222207 = 0x57A66CA93CEA.$$

مطابقت مقدار ذخیره شده با حاصل محاسبه نشان می دهد مسیرهای خواندن از حافظه، اجرای عملیات در ALU و نوشتن نتیجه در حافظه، همگی توسط ریزبرنامه با ترتیب درست کنترل شده اند.



(ب) حاصل ذخیره شده از آدرس 0x10

(آ) دو ورودی و ناحیه خالی نتیجه پیش از اجرا

شکل ۸: مقایسه حافظه پیش و پس از جمع دو عدد ۴۸ بیتی

۸ نتیجه گیری

در این آزمایش، واحد کنترل سخت سیمی کامپیوتر ساده تکمیل شده طی آزمایش های ششم تا هشتم با یک واحد کنترل ریزبرنامه ای جایگزین شد. مسیر داده، فایل ثبات، واحد محاسبات، حافظه های برنامه و داده، قالب دستورهای ماشین و زیرمدار تجزیه دستور بدون تغییر باقی ماندند و فقط روش تولید سیگنال های کنترل تغییر کرد.

ریزحافظه 256×11 توالی واکشی و اجرای دستورهای محاسباتی، دسترسی به حافظه و پرش را نگهداری می کند. شمارنده μPC با استفاده از میدان $\mu Opcode$ و ورودی های شرط، میان افزایش ترتیبی و بارگذاری آدرس پرش انتخاب می کند. همچنین مدار دروازه بندی خروجی تضمین می کند که بیت های آدرس پرش به اشتباه سیگنال کنترلی تولید نکنند.

برای ارزیابی سیستم، دو برنامه مورد انتظار دوباره اجرا شدند. برنامه مجموع ده جمله اول فیبوناچی مقدار 0x58 را در آدرس صفر ذخیره کرد. برنامه جمع چندبیتی نیز به دلیل محدودیت ۳۲ کلمه ای حافظه برنامه برای دو عدد ۴۸ بیتی اجرا شد و حاصل 0x57A66CA93CEA را به درستی از آدرس شانزده قرار داد. هر دو برنامه پس از پایان با پرش بدون شرط به دستور توقف خود بازمی گردند. این نتایج صحت ریزبرنامه های واکشی، تشخیص و اجرای تمام گروه های دستور ماشین را تأیید می کنند.